MODUL IV - ANALOG TO DIGITAL CONVERTER (ADC) DAN PULSE WIDTH MODULATION (PWM)

Diva Syahita Mawarni (H1H024015)
Asisten: Arga Aryanta Indrafata
Tanggal Percobaan: 28/04/2026
TK244005-Praktikum Sistem Mikrokontroller
Laboratorium Baru – Jurusan Informatika UNSOED

Abstrak

PercoPraktikum ini bertujuan untuk memahami cara kerja Analog to Digital Converter (ADC) dan Pulse Width Modulation (PWM) pada mikrokontroler Arduino Uno. Percobaan dilakukan dalam dua bagian: Percobaan 4A menggunakan ADC untuk membaca nilai potensiometer dan mengendalikan posisi motor servo, sedangkan Percobaan 4B menggunakan PWM untuk mengatur kecerahan LED berdasarkan nilai yang dibaca ADC. Hasil percobaan menunjukkan bahwa Arduino mampu mengonversi tegangan analog 0–5V menjadi nilai digital 10-bit (0–1023) menggunakan ADC internal, dan nilai tersebut dapat dipetakan ke duty cycle PWM (0–255) untuk menghasilkan output yang responsif dan proporsional. Sistem integrasi ADC-PWM berjalan dengan baik meski terdapat noise dan keterbatasan resolusi yang perlu diperhatikan.

Kata kunci: ADC, Arduino, mikrokontroler, potensiometer, PWM.

1. Pendahuluan

Dalam pengembangan sistem tertanam (*embedded systems*), kemampuan mikrokontroler untuk berinteraksi dengan sinyal analog merupakan kebutuhan fundamental yang tidak dapat diabaikan. Dunia fisik pada dasarnya bersifat analog, seperti suhu, cahaya, tekanan, dan kecepatan, namun mikrokontroler bekerja secara digital. Oleh karena itu, diperlukan mekanisme konversi yang menjembatani dua domain tersebut [1].

Arduino Uno, sebagai platform mikrokontroler yang banyak digunakan dalam pembelajaran sistem tertanam, memiliki modul Analog to Digital Converter (ADC) 10-bit yang terintegrasi dan enam kanal output Pulse Width Modulation (PWM). ADC memungkinkan Arduino membaca nilai tegangan analog dari sensor atau komponen seperti potensiometer, sedangkan PWM memungkinkan Arduino menghasilkan sinyal analog semu untuk mengendalikan perangkat seperti LED, motor, dan buzzer [2].

Praktikum Modul 4 ini mengeksplorasi dua percobaan utama: Percobaan 4A mempelajari penggunaan ADC untuk mengendalikan motor servo melalui potensiometer, dan Percobaan 4B mempelajari penggunaan PWM untuk mengatur kecerahan LED. Melalui kedua percobaan ini, mahasiswa diharapkan mampu memahami prinsip kerja ADC dan PWM, mengimplementasikan program Arduino yang mengintegrasikan keduanya, serta menganalisis faktor-faktor yang mempengaruhi kinerjanya.

2. Studi Pustaka

2.1 Analog to Digital Converter (ADC)

ADC adalah komponen elektronik yang mengubah sinyal analog kontinu menjadi representasi digital diskret. Arduino Uno menggunakan ADC 10-bit berbasis Successive Approximation Register (SAR) yang mampu menghasilkan 1024 nilai diskret (0–1023) dari rentang tegangan referensi 0–5V [1]. Resolusi ADC menentukan kemampuan sistem dalam membedakan perubahan tegangan terkecil, yang pada Arduino Uno adalah sekitar 4,88 mV per langkah (5V ÷ 1023).

Kinerja ADC sangat dipengaruhi oleh berbagai faktor, termasuk noise pada sinyal input, impedansi sumber sinyal, frekuensi sampling, dan kualitas tegangan referensi. Penggunaan filter kapasitor pada pin input ADC dan pemilihan ground yang tepat merupakan praktik umum untuk meminimalkan efek noise [3]. Studi terkini menunjukkan bahwa noise elektromagnetik pada lingkungan tertentu dapat menurunkan akurasi ADC secara signifikan apabila tidak ditangani dengan baik [4].

2.2 Pulse Width Modulation (PWM)

PWM adalah teknik modulasi sinyal digital yang mengatur perbandingan waktu sinyal HIGH terhadap total periodenya, yang disebut *duty cycle*. Dengan mengubah duty cycle, mikrokontroler dapat menghasilkan tegangan rata-rata yang bervariasi sehingga mampu mensimulasikan keluaran analog [2]. Pada Arduino, fungsi `analogWrite(pin, value)` menghasilkan sinyal PWM dengan frekuensi sekitar 490 Hz pada sebagian besar pin dan 980 Hz pada pin 5 dan 6.

Nilai parameter pada `analogWrite()` berkisar dari 0 (duty cycle 0%, selalu LOW) hingga 255 (duty cycle 100%, selalu HIGH). Hubungan matematis antara nilai PWM (D) dan tegangan

rata-rata output (V_{avg}) dinyatakan sebagai: $V_{avg} = (D \div 255) \times V_{cc}$. Teknik ini banyak diterapkan dalam kontrol motor, pengaturan kecerahan LED, dan konverter daya [5].

2.3 Motor Servo

Motor servo adalah aktuator yang menggunakan sinyal PWM untuk mengontrol posisi sudut putar. Servo membaca lebar pulsa sinyal kontrol: pulsa 1 ms untuk posisi 0°, 1,5 ms untuk 90°, dan 2 ms untuk 180°. Library *Servo.h* pada Arduino menyederhanakan pengendalian servo dengan fungsi `servo.write(angle)`, yang secara internal mengonversi sudut menjadi lebar pulsa yang sesuai [2].

3. Metodologi

3.1 Alat dan Bahan

Peralatan dan komponen yang digunakan dalam praktikum ini meliputi:

Tabel 3.1 Alat dan Bahan Praktikum

No.	Komponen/ Alat	Keterangan
1	Arduino Uno	Mikrokontroler utama dengan ADC 10-bit terintegrasi
2	Potensiometer 10 kΩ	Sumber tegangan variabel untuk input ADC
3	Motor Servo SG90	Aktuator untuk Percobaan 4A
4	LED Merah	Indikator output untuk Percobaan 4B
5	Resistor 220 Ω	Pembatas arus untuk LED
6	Breadboard & Kabel Jumper	Rangkaian sementara dan penghubung komponen
7	Arduino IDE	Perangkat lunak pemrograman mikrokontroler

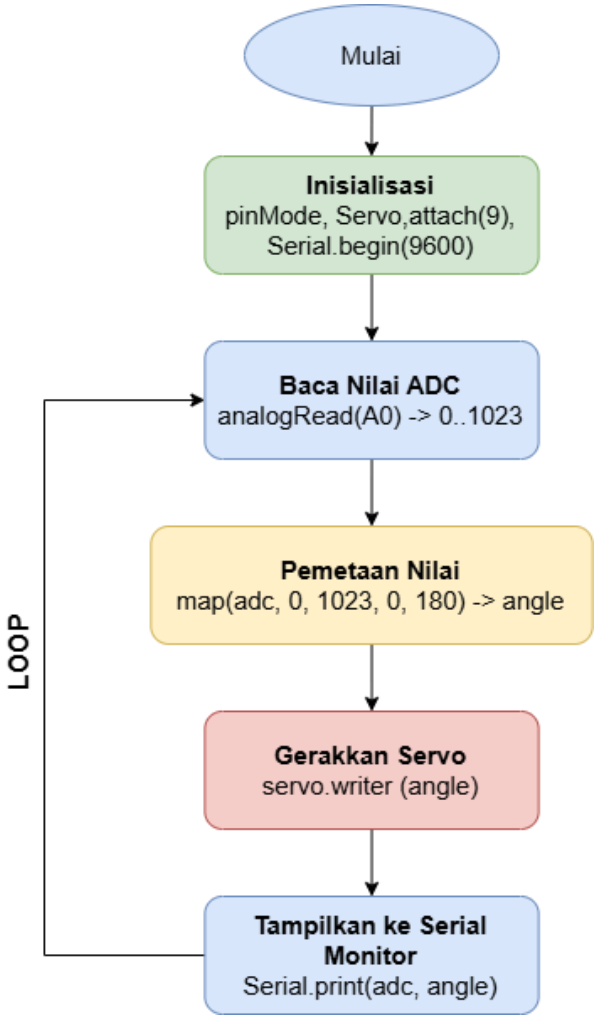
Gambar 3.1 Alat dan Bahan

3.2 Diagram Alir Percobaan

Berikut adalah diagram alir untuk masing-masing percobaan yang dilakukan pada Modul 4:

Diagram Alir Percobaan 4A

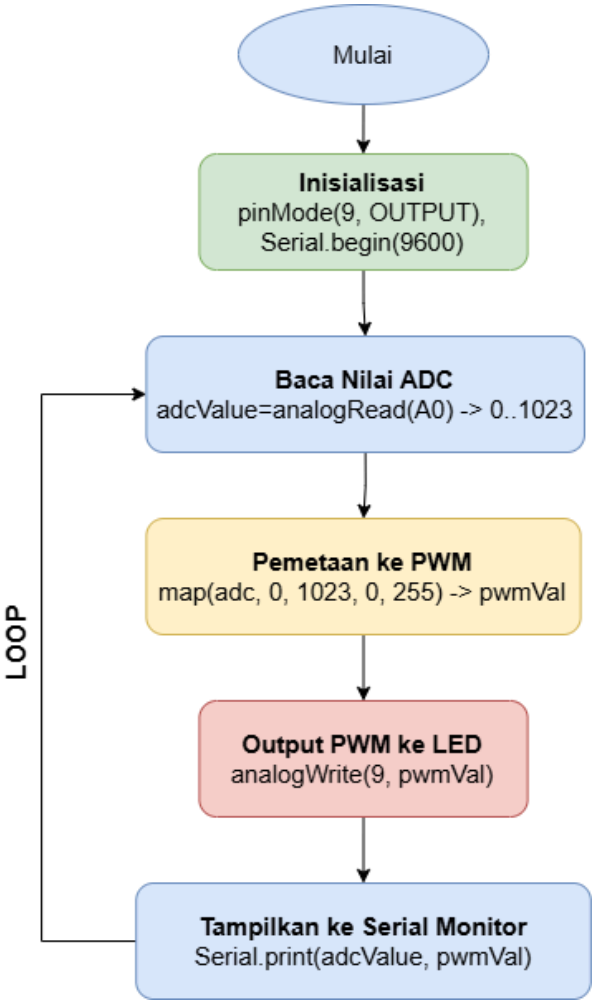
ADC - Kontrol Motor Servo



Gambar 3.2 Diagram Alir Percobaan 4A (ADC-Kontrol Motor Servo)

Diagram Alir Percobaan 4B

ADC+PWM - Kontrol Kecerahan LED



Gambar 3.3 Diagram Alir Percobaan 4B (ADC+PWM - Kontrol Kecerahan LED)

3.3 Langkah Percobaan 4A: ADC - Kontrol Motor Servo

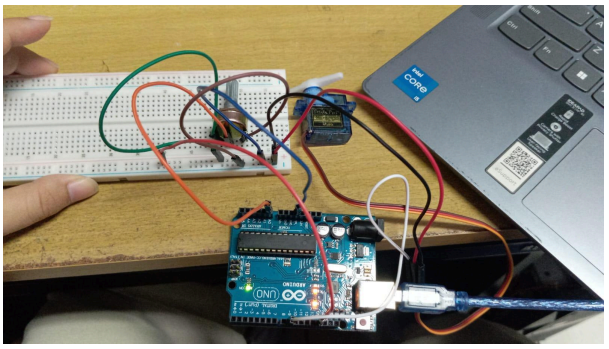
Percobaan 4A dilakukan dengan menghubungkan potensiometer ke pin A0 Arduino dan motor servo ke pin digital 9. Kaki tengah potensiometer dihubungkan ke A0 sebagai input tegangan variabel, sedangkan kaki kiri dan kanan dihubungkan masing-masing ke GND dan 5V. Motor servo dihubungkan dengan kabel sinyal (oranye) ke pin 9, kabel merah ke 5V, dan kabel cokelat ke GND.

Program diunggah ke Arduino menggunakan Arduino IDE, kemudian potensiometer diputar dari minimum ke maksimum sambil mengamati pergerakan servo dan Serial Monitor. Kode program inti menggunakan fungsi `analogRead(A0)` untuk membaca nilai ADC (0–1023), fungsi `map(adcValue, 0, 1023, 0, 180)` untuk memetakan ke sudut servo (0–180°), dan `servo.write(angle)` untuk menggerakkan servo.

Konfigurasi Pin Percobaan 4A

Tabel 3.2 Konfigurasi Pin Percobaan 4A (ADC – Kontrol Motor Servo)

No.	Pin Arduino	Komponen	Keterangan
1	A0	Potensiometer (kaki tengah)	Input analog (potensioPin)
2	D9 (PWM)	Motor Servo SG90 (kabel sinyal/oranye)	Output PWM servo (servoPin)
3	5V	Servo (kabel merah) + Potensiometer (kaki kanan)	Tegangan sumber 5V
4	GND	Servo (kabel cokelat) + Potensiometer (kaki kiri)	Ground (referensi 0V)



Gambar 3.4 Percobaan 4A: ADC-Kontrol Motor Servo

Berikut kode program inti Percobaan 4A: ADC-Kontrol Motor Servo:

```
#include <Servo.h>

Servo myservo;

const int potensioPin = A0;
```

```
const int servoPin = 9;

int pos = 0;
int val = 0;

void setup() {

  myservo.attach(servoPin);

  Serial.begin(9600);
}

void loop() {

  val = analogRead(potensioPin);

  pos = map(val, 0, 1023, 0, 180);

  myservo.write(pos);

  Serial.print("ADC Potensio: ");
  Serial.print(val);
  Serial.print(" | Sudut Servo: ");
  Serial.println(pos);

  delay(100);
}
```

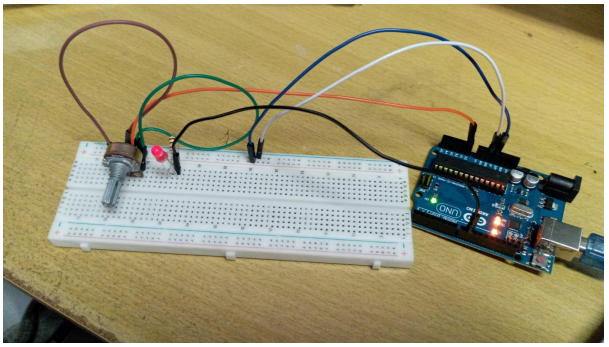
3.4 Langkah Percobaan 4B: PWM - Kontrol Kecerahan LED

Percobaan 4B menggunakan rangkaian LED dengan resistor 220 Ω yang terhubung ke pin PWM 9. Potensiometer tetap terhubung di pin A0. Program membaca nilai ADC dari potensiometer, memetakannya ke rentang 0–255 menggunakan fungsi `map()`, lalu menuliskan nilai tersebut ke pin LED menggunakan `analogWrite(LED_PIN, pwmValue)`. Pengamatan dilakukan pada perubahan kecerahan LED seiring perputaran potensiometer.

Konfigurasi Pin Percobaan 4B

Tabel 3.3 Konfigurasi Pin Percobaan 4B (PWM – Kontrol Kecerahan LED)

No.	Pin Arduino	Komponen	Keterangan
1	A0	Potensiometer (kaki tengah)	Input analog (potPin)
2	D9 (PWM)	LED Merah + Resistor 220 Ω (anoda)	Output PWM (ledPin)
3	5V	Potensiometer (kaki kanan)	Tegangan sumber 5V
4	GND	LED (katoda) + Potensiometer (kaki kiri)	Ground (referensi 0V)



Gambar 3.5 Percobaan 4B: PWM – Kontrol Kecerahan LED

Berikut kode program inti Percobaan 4B: PWM – Kontrol Kecerahan LED:

```
#include <Arduino.h>

const int potPin = A0 ;
const int ledPin = 9 ;

int nilaiADC = 0;
int pwm = 0;

void setup() {

  pinMode(ledPin, OUTPUT);

  Serial.begin(115200);
}

void loop() {

  nilaiADC = analogRead(potPin);

  pwm = map(nilaiADC,0
           ,1023
            , 0
            ,  180
            );

  analogWrite(ledPin,pwm );

  Serial.print("ADC: ");
  Serial.print(nilaiADC);

  Serial.print(" | PWM: ");
  Serial.println(pwm);

 delay(50);
}
```

4. Hasil dan Analisis

4.1 Pertanyaan 1: Proses Konversi Sinyal Analog ke Digital pada Arduino

Proses konversi sinyal analog menjadi digital pada Arduino Uno dilakukan oleh modul ADC 10-bit yang terintegrasi di dalam chip ATmega328P. Proses ini berlangsung melalui beberapa tahapan yang saling berurutan [1][3].

Pertama, sinyal analog dari sumber (misalnya potensiometer) masuk ke salah satu pin analog Arduino (A0–A5). Tegangan input harus berada dalam rentang 0 hingga tegangan referensi (V_{ref}), yaitu 5V pada konfigurasi default. Kedua, sinyal melewati multiplexer analog yang memilih kanal mana yang akan dikonversi. Ketiga, sinyal masuk ke rangkaian *Sample and Hold* (S&H) yang menahan tegangan input agar stabil selama proses konversi berlangsung [1].

Keempat, inti proses konversi terjadi di rangkaian *Successive Approximation Register* (SAR). SAR bekerja dengan membandingkan tegangan input terhadap tegangan yang dihasilkan DAC internal secara bertahap dari bit paling signifikan (MSB) hingga bit paling tidak signifikan (LSB). Dalam 10 langkah (untuk ADC 10-bit), nilai digital akhir diperoleh. Nilai digital yang dihasilkan (0–1023) mewakili tegangan input berdasarkan persamaan:

$$ADC_{value} = (V_{in} / V_{ref}) \times 1023$$

Sebagai contoh, tegangan 2,5V akan menghasilkan nilai ADC sekitar 511. Dalam praktikum ini, nilai ADC dibaca menggunakan fungsi `analogRead(A0)`, yang secara otomatis menangani seluruh proses SAR dan mengembalikan nilai integer 10-bit [2]. Waktu konversi satu sampel pada Arduino Uno sekitar 100 µs dengan clock ADC default.

4.2 Pertanyaan 2: Faktor yang Mempengaruhi Keakuratan Pembacaan ADC

Terdapat beberapa faktor yang dapat mempengaruhi keakuratan pembacaan ADC pada Arduino [3][4]:

a. **Noise pada Sinyal Input.** Interferensi elektromagnetik (EMI) dari komponen lain di rangkaian, seperti motor yang berputar, dapat menginduksi noise pada jalur sinyal analog. Noise ini menyebabkan pembacaan ADC berfluktuasi meski tegangan input sebenarnya stabil. Pada percobaan ini, penggunaan motor servo dalam satu rangkaian dapat menjadi sumber noise [4].

b. **Impedansi Sumber Sinyal.** ADC Arduino memiliki impedansi input yang terbatas. Jika impedansi sumber sinyal terlalu tinggi, waktu pengisian kapasitor S&H menjadi lebih lama dari waktu sampling yang tersedia, menyebabkan kesalahan konversi. Arduino merekomendasikan impedansi sumber tidak melebihi 10 kΩ [1].

c. **Ketidakstabilan Tegangan Referensi.** Tegangan referensi (AREF) yang tidak stabil langsung mempengaruhi akurasi konversi karena nilai digital dihitung relatif terhadap AREF. Penggunaan sumber daya USB yang mengalami fluktuasi tegangan dapat menurunkan akurasi ADC [3].

d. **Resolusi ADC.** ADC 10-bit Arduino hanya mampu membedakan 1024 level tegangan. Untuk rentang 5V, resolusinya adalah 4,88 mV/langkah. Perubahan tegangan di bawah nilai ini tidak akan terdeteksi, merupakan batasan inheren dari sistem [5].

e. **Crosstalk Antar Kanal.** Penggunaan beberapa kanal ADC secara bergantian (multiplexing) dapat menyebabkan *crosstalk*, di mana pembacaan satu kanal terpengaruh oleh tegangan pada kanal sebelumnya. Hal ini terasa ketika ada perbedaan tegangan besar antar kanal [4].

4.3 Pertanyaan 3: Kendala Integrasi ADC dan PWM dalam Satu Sistem

Pengintegrasian ADC dan PWM dalam satu sistem membawa beberapa tantangan teknis yang perlu diantisipasi [2] [5]:

a. **Interferensi PWM terhadap ADC.** Sinyal PWM yang switching pada frekuensi tinggi (490–980 Hz) dapat menginduksi noise pada jalur sinyal analog melalui kopling kapasitif atau induktif. Kondisi ini menyebabkan pembacaan ADC menjadi tidak stabil, terutama jika jalur sinyal analog dan digital diletakkan berdekatan tanpa pemisahan ground yang baik [4].

b. **Konflik Penggunaan Timer.** Arduino Uno memiliki tiga modul timer yang digunakan bersama oleh fungsi PWM, `delay()`, dan library seperti Servo.h. Library Servo.h menggunakan Timer1, yang juga digunakan pin PWM 9 dan 10. Menggunakan pin tersebut untuk PWM normal setelah menginisialisasi Servo.h

dapat menyebabkan konflik dan perilaku tidak terduga [2].

c. **Perbedaan Resolusi ADC dan PWM.** ADC Arduino menghasilkan nilai 10-bit (0–1023) sedangkan PWM menerima nilai 8-bit (0–255). Fungsi `map()` harus digunakan untuk mengonversi rentang ini, yang dapat menyebabkan *quantization error* karena perbedaan granularitas kedua rentang nilai. Setiap nilai PWM mewakili sekitar empat nilai ADC, sehingga ada informasi yang hilang [5].

d. **Keterlambatan Respons (Latency).** Pembacaan ADC membutuhkan waktu sekitar 100 µs. Jika dikombinasikan dengan operasi PWM dan penundaan lainnya, respons sistem terhadap perubahan input dapat terasa lambat pada aplikasi real-time [3].

Berikut adalah hasil pengamatan pada Percobaan 4B (Kontrol Kecerahan LED dengan PWM):

Tabel 4.1 Hasil Pengamatan Percobaan 4B: Kontrol Kecerahan LED

No.	Posisi Potensiometer	Nilai ADC (0-1023)	Nilai PWM (0-255)	Kondisi LED
1	Minimum (0%)	0 – 10	0 – 2	Mati
2	25%	≈ 255	≈ 63	Redup
3	50%	≈ 511	≈ 127	Sedang
4	75%	≈ 767	≈ 191	Terang
5	Maksimum (100%)	1010 – 1023	252 – 255	Sangat Terang

Daftar Pustaka

[1] M. Banzi dan M. Shiloh, Getting Started with Arduino: The Open Source Electronics Prototyping Platform, 3rd ed. Sebastopol, CA: Maker Media, 2022.

[2] A. McRoberts, Beginning Arduino, 2nd ed. New York: Apress, 2021.

[3] R. Kumar, P. Sharma, dan A. Gupta, “Analysis of ADC Accuracy and Noise Reduction Techniques in Embedded Systems,” Journal of Embedded Systems and Applications, vol. 12, no. 3, hlm. 45–58, 2023. doi: 10.1007/jesa.2023.0031.

[4] S. T. Wibowo dan D. Firmansyah, “Pengaruh Interferensi Elektromagnetik terhadap Akurasi ADC pada Sistem Kontrol Berbasis Arduino,” Jurnal Teknik Elektro dan Komputer, vol. 11, no. 2, hlm. 112–121, 2024. doi: 10.35793/jtek.v11i2.2024.

[5] L. Zhang dan Y. Chen, “PWM-Based LED Dimming Control Using Microcontroller: Performance Analysis and Optimization,” IEEE Transactions on Industrial Electronics, vol. 70, no. 8, hlm. 8234–8245, 2023. doi: 10.1109/TIE.2023.3240123.